

SBlеPay REST Assured Framework

Complete Reasoning Guide

WHAT • WHY • HOW — For Every Concept & Every File

From Zero to Enterprise-Grade API Automation

Building Deep Understanding, Not Just Code

Version 1.0 • 59 Files • 31+ Tests • 9 APIs • Full CI/CD

Table of Contents

Part I — Foundation Concepts

1. What Is API Testing & Why REST Assured?
2. HTTP Fundamentals You Must Know
3. JSON — The Language APIs Speak
4. REST Assured given()-when()-then() Deep Dive
5. Maven — Why & How It Manages Your Project

Part II — Framework Architecture (Reasoning)

6. Layered Architecture — Why Separate Concerns?
7. Configuration Management — The Owner Pattern
8. The API Client — Central HTTP Brain
9. Authentication — Dual Token System Explained
10. Encryption — AES/CBC Step by Step
11. POJO Models — Type Safety Over Strings
12. Service Layer — The Business Logic Shield
13. Response Validation — The Fluent Chain
14. Data Generation — Never Hardcode Again
15. JSON Templates — External Payload Management

Part III — Testing Concepts (Reasoning)

16. TestNG Fundamentals for API Testing
17. Soft vs Hard Assertions — Complete Comparison
18. Test Groups, Suites & Parallel Execution
19. Retry & Flaky API Handling
20. E2E Flow — Chain APIs Like Production

Part IV — Infrastructure (Reasoning)

21. Logging — Seeing Without Exposing
22. Reporting — Extent Reports Integration
23. CI/CD — Jenkins, GitHub Actions, Docker

Part V — Every Framework File Explained

24. File-by-File Walkthrough (All 59 Files)

Part VI — Payload Management (8 Approaches)

Part VII — Interview Questions & Anti-Patterns

Part I — Foundation Concepts

1. What Is API Testing & Why REST Assured?

WHAT: API testing means sending HTTP requests to a server and verifying the responses **without a browser or UI**. You talk directly to the backend. Think of it as texting the kitchen directly instead of telling the waiter.

WHY: UI tests are slow (open browser, click, wait, render). API tests are fast (send JSON, get JSON, validate). In a payment gateway like SBlePay, 90% of the business logic lives in the API layer. If Customer Create returns wrong data, no amount of UI testing will catch the root cause. API testing catches bugs **closer to where they originate**.

HOW: You use a library like REST Assured to construct an HTTP request (method + URL + headers + body), send it to the server, receive the response (status code + headers + body), and then assert that the response matches your expectations.

THINK: Ask yourself: 'If I remove the UI completely, does the business logic still work?' That is what API testing answers. In SBlePay, the mobile app and web portal both call the same APIs. Testing the API once covers both frontends.

1.1 Why REST Assured Over Alternatives?

You already know Java and Selenium. REST Assured was designed to feel natural to Java developers. Here is the reasoning behind choosing it:

Factor	REST Assured	Apache HttpClient	OkHttp	Postman
Built for testing?	Yes — given/when/then DSL	No — general HTTP client	No — general HTTP client	Manual — GUI tool
JSON parsing built-in?	Yes — auto-detects Jackson	No — add manually	No — add manually	Limited scripting
Schema validation?	Yes — module available	No	No	No
Integrates with TestNG?	Seamless	You wire it yourself	You wire it yourself	Newman (separate)
Learning curve	Low (you know Java)	High (verbose)	Medium	Low but not code-first
CI/CD friendly?	Yes — Maven + TestNG	Yes but more setup	Yes but more setup	Needs Newman CLI

THINK: The key reasoning: REST Assured is the **ONLY** library purpose-built for API **testing**. HttpClient and OkHttp are for **building** HTTP clients in production apps. Using them for testing means writing 3x more boilerplate. REST Assured gives you assertions, logging, and serialisation out of the box.

2. HTTP Fundamentals You Must Know

WHAT: HTTP (HyperText Transfer Protocol) is the standard way clients (your tests) talk to servers (SBlePay API). Every API call is an HTTP request-response cycle.

WHY: Without understanding HTTP, you will not understand why a test fails. 'Got 401' means nothing unless you know 401 = Unauthorized = your token is wrong/expired/missing. 'Got 403' means Forbidden = your token is valid but does not have permission for this action.

2.1 HTTP Methods — Step by Step Reasoning

Each method has a **semantic meaning**. The server expects you to use the right one:

Method	What It Does	SBlеPay Example	Why This Method?
POST	Create or submit data	Create Customer, Create Order, Payment	You are SENDING data to create something new
GET	Retrieve data (read-only)	Retrieve Customer by ID	You are READING data without changing anything
PUT	Replace entire resource	Not used in SBlеPay	Full replacement (all fields required)
PATCH	Update part of resource	Not used in SBlеPay	Partial update (only changed fields)
DELETE	Remove resource	Not directly (uses status update)	Remove a resource permanently

THINK: SBlеPay uses POST for almost everything — even status updates. This is common in payment gateways because every action creates an **audit trail** (a new record). GET is used only for reading customer data. This design choice prioritises traceability over REST purity.

2.2 HTTP Status Codes — What They Really Mean

Status codes tell you WHAT happened at the HTTP level, BEFORE you even look at the body:

Code	Name	What It Means	Your Test Should Do
200	OK	Server processed your request successfully	Check the body — business logic may still have failed (status:0)
201	Created	Resource was created	Verify the new resource ID in response
400	Bad Request	Your request format is wrong (malformed JSON, missing Content-Type)	Check your payload construction
401	Unauthorized	No token OR token is invalid/expired	Check TokenManager — is token being sent? Is it expired?
403	Forbidden	Token is valid but NOT allowed for this action	You are using ACCESS token where TXN token is needed (or vice versa)
404	Not Found	The URL/endpoint does not exist	Check your endpoint constant
500	Internal Server Error	Server crashed while processing	Log everything, report as server bug
503	Service Unavailable	Server is temporarily down	Retry with exponential backoff

THINK: CRITICAL REASONING for SBlеPay: A 200 status does NOT always mean success! SBlеPay returns HTTP 200 even for business errors. The body will have status:0 with error codes. This is why you MUST check both the HTTP status AND the business status field. Many beginners only check HTTP 200 and miss actual failures.

2.3 HTTP Headers — Metadata About Your Request

WHAT: Headers are key-value pairs sent with every request/response. They carry metadata: authentication tokens, content format, tracing IDs.

SBlеPay headers and why each exists:

Header	Value Example	Why It Exists	Who Sends It
Content-Type	application/json	Tells server: 'My body is JSON, parse it as JSON'	Your test (client)
Authorization	Bearer eyJhbG...	Carries the JWT token for authentication	Your test (client)
Merchant-API-Key-Id	1000003	Identifies WHICH merchant is calling	Your test (token generation only)
Merchant-API-Key-Secret	secret123	Proves the merchant's identity (like a password)	Your test (token generation only)
x-referrer	website URL	Required for Transaction Token — identifies source	Your test (txn token only)
X-Correlation-ID	uuid-string	Our custom header to trace requests in server logs	Our framework (auto-added)

3. JSON — The Language APIs Speak

WHAT: JSON (JavaScript Object Notation) is the data format used by SBlePay APIs. Both request bodies and response bodies are JSON. It is human-readable text that represents structured data using key-value pairs, arrays, and nested objects.

WHY: You need to understand JSON deeply because: (1) You BUILD request payloads as JSON, (2) You PARSE response JSON to extract values, (3) You VALIDATE response JSON against expected schemas, (4) You MODIFY JSON for negative tests (remove fields, corrupt values).

3.1 JSON Data Types — Know Every One

Type	Example	Java Equivalent	SBlePay Usage
String	"customerName": "John"	String	Names, emails, IDs, tokens
Number (int)	"status": 1	int / Integer	Business status (1=success, 0=error)
Number (decimal)	"amount": 500.00	double / BigDecimal	Order amounts, fees
Boolean	"active": true	boolean / Boolean	Flags (not used much in SBlePay)
Null	"gstNumber": null	null	Optional fields when not provided
Array	"data": [...]	List<>	Response data array, errors array
Object	"address": {...}	POJO / Map	Nested structures like deviceDetail

3.2 JsonPath — Navigating JSON Responses

WHAT: JsonPath is a query language for extracting values from JSON, like XPath for XML. REST Assured includes it built-in.

Given this SBlePay response:

```
{"status":1,"data":[{"customerId":"C001","customerName":"Priya","status":"ACTIVE"}]}
```

Here is how you navigate it:

JsonPath Expression	Returns	Reasoning
status	1	Top-level field, direct access
data[0].customerId	"C001"	First element of data array, then customerId field
data[0].status	"ACTIVE"	First element, status field (NOT the top-level status!)
data.size()	1	How many items in the data array
errors[0].errorCode	"2001"	For error responses — first error's code

THINK: Notice 'status' vs 'data[0].status' — they are DIFFERENT fields. The top-level 'status' is the business success/failure flag (1 or 0). The nested 'data[0].status' is the customer/order status (ACTIVE, CREATED, etc). Confusing these is a common beginner mistake.

4. REST Assured given()-when()-then() Deep Dive

WHAT: This is the BDD (Behaviour-Driven Development) pattern that structures every REST Assured test into three clear phases: ARRANGE (given), ACT (when), ASSERT (then).

WHY: If you know Selenium, you know tests get messy fast. given/when/then forces structure. Anyone reading the test immediately understands: what is being set up, what action is performed, and what is being checked. This is not just style — it is a communication tool for your team.

HOW: Each phase maps to a specific responsibility:

Phase	REST Assured Method	Responsibility	SBlePay Example
ARRANGE	given()	Set up: base URI, headers, auth, body, query params, filters	Set Bearer token, Content-Type, encrypted payload
ACT	when()	Execute: HTTP method + endpoint	.post('/transaction/v1/customer/create')
ASSERT	then()	Validate: status code, body fields, headers, schema, timing	.statusCode(200).body('status', equalTo(1))

4.1 Step-by-Step: What Happens Inside a REST Assured Call

When you write this code, here is EXACTLY what happens internally, step by step:

```
given().header('Authorization', 'Bearer xxx').contentType(JSON).body(payload).when().post('/customer/create').then().statusCode(200);
```

- **Step 1 — given():** Creates a RequestSpecification object. Think of it as a blank form waiting to be filled.
- **Step 2 — .header(...):** Adds the Authorization header to the form. This will be sent as an HTTP header.
- **Step 3 — .contentType(JSON):** Adds 'Content-Type: application/json' header. Tells the server your body is JSON.
- **Step 4 — .body(payload):** Jackson serializes your Java object to a JSON string. This becomes the HTTP request body.
- **Step 5 — when():** Transitions from setup to execution phase. Returns a RequestSender.
- **Step 6 — .post('/customer/create'):** NOW the HTTP call actually happens. REST Assured sends the request and waits for the response.
- **Step 7 — then():** Wraps the response for validation. Returns a ValidatableResponse.
- **Step 8 — .statusCode(200):** Checks response.getStatusCode() == 200. Fails immediately if not (hard assert).

THINK: The KEY insight: nothing happens until .post() is called. Everything before it is just building the request. This is the Builder Pattern in action — you configure, configure, configure, then execute once.

4.2 extract() — Getting the Response Object

WHAT: Sometimes you need the raw Response object (not just inline assertions). extract() gives you that.

WHY: In our framework, Service methods return the Response object so test methods can validate using ResponseValidator. This separates the 'making the call' from 'checking the result'.

```
Response response = given()...when().post('/endpoint').then().extract().response();
```

```
String customerId = response.jsonPath().getString('data[0].customerId');  
int statusCode = response.getStatusCode();
```

5. Maven — Why & How It Manages Your Project

WHAT: Maven is a build tool that manages your project's dependencies (libraries), compilation, testing, and packaging. The pom.xml file is the single source of truth for your project configuration.

WHY: Without Maven, you would manually download 20+ JAR files (REST Assured, TestNG, Jackson, Lombok, Log4j2, ...), put them in a /lib folder, manage version conflicts yourself, and write custom scripts to compile and run tests. Maven automates ALL of this.

HOW: You declare what you need in pom.xml. Maven downloads it from a central repository, resolves transitive dependencies (libraries that your libraries need), and makes everything available to your code.

5.1 Key Dependencies — Why Each One Exists

Dependency	What It Does	Why We Need It (Reasoning)
rest-assured	API testing DSL	The core library. Without this, there is no framework.
testng	Test execution framework	Annotations (@Test), suites, groups, parallel execution, listeners. JUnit alternative.
jackson-databind	JSON serialization	Converts POJO to JSON and back. REST Assured auto-detects it.
lombok	Reduces boilerplate	@Data generates getters/setters. @Builder generates builder pattern. @Slf4j adds logger. Saves 1000+ lines.
log4j2	Logging framework	Professional logging with levels, file output, rolling archives. System.out.println is not enterprise.
extentreports	HTML test reports	Stakeholders cannot read console output. They need visual reports with charts.
assertj-core	Fluent assertions	Better error messages than TestNG asserts. Soft assertion support. Chain syntax.
owner	Type-safe config	No more getProperty('base.url') with typo risk. config.baseUrl() is compile-safe.
javafaker	Random test data	Generates realistic names, phones, emails. No hardcoded 'John Doe' everywhere.
poi	Excel read/write	Data-driven testing from Excel spreadsheets.
bcprov	Cryptography (BouncyCastle)	AES encryption for SBlePay's encrypted request payloads.
json-schema-validator	Schema validation	Validate response structure against a JSON Schema contract.

THINK: REASONING for version choices: We use the LATEST stable versions (REST Assured 5.4.0, TestNG 7.9.0, Jackson 2.17.0). Older versions may have security vulnerabilities or missing features. In enterprise projects, staying current is a security requirement.

5.2 Maven Profiles — One Project. Multiple Modes

WHAT: Profiles let you switch behaviour without changing code. We have environment profiles (sit/uat/prod) and suite profiles (smoke/regression/e2e).

WHY: In an enterprise, you test against SIT daily, UAT before releases, and prod for smoke checks. Each environment has different URLs and credentials. Profiles let you switch with a flag: mvn test -Psit

-Psmoke.

HOW: Each profile sets a system property. The framework reads it at runtime: if env=sit, load sit.properties. If suite=smoke, use testng-smoke.xml.

Part II — Framework Architecture (Reasoning)

6. Layered Architecture — Why Separate Concerns?

WHAT: Our framework has 4 layers: Test Layer (top) → Service Layer → Core Layer → Support Layer (bottom). Each layer has one job and talks only to the layer below it.

WHY: Imagine you change the encryption algorithm from AES-128 to AES-256. With layered architecture: you change ONE file (EncryptionUtil.java). Services still call encrypt(). Tests don't even know encryption exists. Without layers: you change 50 test files. This is the **Single Responsibility Principle** in action.

Layer	Contains	Knows About	Does NOT Know About
Test Layer	AccessTokenTest, CustomerTest, OrderTest, E2E	Service methods, ResponseValidator	HTTP, encryption, headers, tokens
Service Layer	CustomerService, OrderService, TransactionService	ApiClient, TokenManager, EncryptionUtil, POJOs	Test assertions, report logging
Core Layer	ApiClient, TokenManager, EncryptionUtil, ResponseValidator	REST Assured, config, HTTP details	Business rules, test data
Support Layer	ConfigFactory, DataGenerator, JsonUtil, ExtentManager	File system, properties, randomness	APIs, HTTP, business logic

THINK: The REASONING TEST: Can you change the base URL without touching any test file? YES — change config.properties. Can you change encryption without touching any test file? YES — change EncryptionUtil. Can you add a new API without modifying existing tests? YES — add a new Service + new Test. If the answer is always YES, your architecture is correct.

7. Configuration Management — The Owner Pattern

WHAT: The Owner library creates a Java interface backed by a properties file. Instead of Properties.getProperty('base.url') (returns String, could be null, typo-prone), you call config.baseUrl() (returns String, compile-time checked, documented in the interface).

WHY: In production frameworks: (1) Property keys get mistyped — 'base_url' vs 'base.url' vs 'baseUrl'. Owner catches this at compile time. (2) Type conversion is manual — Integer.parseInt(props.getProperty('timeout')). Owner does this automatically. (3) Environment switching needs custom code. Owner supports @Config.Sources with variable substitution.

HOW: Step 1: Define an interface with methods matching your properties. Step 2: Annotate with @Config.Sources pointing to your files. Step 3: Use ConfigFactory.create() to get a type-safe instance. The Owner library maps method names to property keys automatically.

```
// FrameworkConfig.java (the interface)
@Config.Sources({'file:src/main/resources/environments/${env}.properties',
'file:src/main/resources/config/config.properties'})
public interface FrameworkConfig extends Config {
```

```

@Key('base.url') String baseUrl();
@Key('retry.max.count') @DefaultValue('2') int retryMaxCount();
}

// Usage anywhere in framework
String url = ConfigFactory.getConfig().baseUrl(); // type-safe, no casting

```

THINK: REASONING: Why not just use Properties? Because Properties.getProperty() returns String or null. You discover typos at RUNTIME when a test fails. With Owner, a method name change shows up as a COMPILE ERROR in your IDE immediately. In a 59-file framework, this prevents hours of debugging.

8. The API Client — Central HTTP Brain

WHAT: ApiClient.java is the single class that wraps all REST Assured HTTP operations. Every API call in the framework goes through this class. It creates the base RequestSpecification, adds filters for logging, and provides typed methods: post(), get(), postWithAccessToken(), postWithTxnToken().

WHY: Without a central client: every Service class creates its own given().baseUrl().contentType()... setup. If you need to add a global header (like X-Correlation-ID), you edit every Service. With ApiClient: you add it once, and every API call gets it automatically.

HOW: Step-by-step flow when you call apiClient.postWithAccessToken(endpoint, body):

- Step 1: Get the base RequestSpecification (baseUrl + contentType + filters already set)
- Step 2: Get the access token from TokenManager (cached, auto-refreshed if expired)
- Step 3: Add Authorization: Bearer {token} header
- Step 4: Add X-Correlation-ID: {UUID} header for traceability
- Step 5: Set the request body (Jackson serialises the Java object to JSON)
- Step 6: Execute the POST to the given endpoint
- Step 7: RequestLoggingFilter logs the request (with sensitive data masked)
- Step 8: ResponseLoggingFilter logs the response
- Step 9: Return the raw Response object to the caller

THINK: REASONING: The ApiClient is like a gatekeeper. Nothing gets in or out without going through it. This means: (1) Every request is logged automatically. (2) Every request has a correlation ID. (3) Auth is injected consistently. (4) Timeouts are enforced globally. Change one place, affect all API calls.

9. Authentication — Dual Token System Explained

WHAT: SBlePay uses TWO different JWT tokens with different scopes and lifetimes. The ACCESS token is for merchant operations (customers, orders). The TRANSACTION token is for payment operations (booking, payment initiation). Our TokenManager handles both, with caching and auto-refresh.

WHY: SECURITY REASONING: Why two tokens? Imagine a merchant's ACCESS token leaks. The attacker can view customer data and create orders, but they CANNOT initiate payments or book transactions. The TRANSACTION token is order-specific — it works only for ONE order and expires quickly. This is the **Principle of Least Privilege**.

9.1 Token Lifecycle — Step by Step

Step	Action	Who Does It	Result
1	Test calls <code>customerService.createRandomCustomer()</code>	Test Layer	Service needs an access token
2	Service calls <code>apiClient.postWithAccessToken()</code>	Service Layer	ApiClient asks TokenManager for token
3	TokenManager checks: is cached token still valid?	Core Layer	If valid and not expiring soon, return cached token
4	If expired: TokenManager calls <code>/token/access</code> with merchant credentials	Core Layer	Gets new JWT, caches it, returns it
5	ApiClient adds Bearer header with the token	Core Layer	Request is now authenticated
6	For payment: TokenManager calls <code>/token/txn/{orderHash}</code>	Core Layer	Gets TXN token + hval, caches per orderHash

THINK: REASONING for caching: Without caching, 31 tests = 31 token requests. With caching, 31 tests = 1 token request (if they all run within the token's lifetime). This is faster AND reduces load on the auth server. The cache includes a buffer (60 seconds before expiry) to prevent 'token expired during request' race conditions.

10. Encryption — AES/CBC Step by Step

WHAT: SBlePay encrypts sensitive request payloads (customer data, order data, payment card details) using AES (Advanced Encryption Standard) with CBC (Cipher Block Chaining) mode and PKCS5Padding.

WHY: SECURITY REASONING: API calls travel over the network. Even with HTTPS, the server might log request bodies. Encryption means even if logs are accessed, sensitive data (card numbers, customer details) is unreadable without the key.

HOW: The encryption flow, step by step:

- **Step 1:** Build the plain JSON payload (CustomerCreateRequest serialised to JSON string).
- **Step 2:** Derive a 256-bit AES key from the encryption key string (using SHA-256 hash).
- **Step 3:** Generate a random 16-byte IV (Initialization Vector) for CBC mode.
- **Step 4:** Encrypt the JSON string using AES/CBC/PKCS5Padding with the key and IV.
- **Step 5:** Prepend the IV to the encrypted bytes (receiver needs IV to decrypt).

- **Step 6:** Base64-encode the result (binary data cannot be sent as JSON).
- **Step 7:** Wrap in EncryptedRequestWrapper: {encryptRequest: 'base64string', key: 'hval'}.
- **Step 8:** Send the wrapper as the HTTP body.

THINK: REASONING for CBC mode: ECB (Electronic Codebook) encrypts each block independently, so identical blocks produce identical ciphertext — patterns are visible. CBC chains each block to the previous one, so identical data produces different ciphertext every time. This is why we need an IV (to start the chain).

11. POJO Models — Type Safety Over Strings

WHAT: POJO = Plain Old Java Object. We create Java classes (CustomerCreateRequest, OrderCreateRequest, etc.) that represent the shape of API request/response bodies. Lombok @Data generates getters/setters and @Builder generates the builder pattern.

WHY: REASONING: Compare these two approaches:

```
// Approach A: HashMap (no type safety)
Map body = new HashMap();
body.put('custmerName', 'John'); // TYPO! 'custmerName' instead of 'customerName'
// Compiles fine. Fails at runtime. Hard to debug.

// Approach B: POJO + Builder (type safety)
CustomerCreateRequest.builder()
    .custmerName('John') // COMPILE ERROR! IDE shows red immediately.
    .build();
```

The HashMap approach has a typo that compiles fine but fails at runtime with a confusing error ('required field missing'). The POJO approach catches the typo at COMPILE TIME — your IDE underlines it in red before you even run the test.

THINK: REASONING: In a framework with 31+ tests calling 9 APIs, a typo in a field name could waste hours of debugging. With POJOs, the IDE gives you autocomplete for every field. Rename a field? Right-click Refactor → Rename — every usage updates automatically. This is why every enterprise framework uses POJOs.

12. Service Layer — The Business Logic Shield

WHAT: Service classes (CustomerService, OrderService, TransactionService) sit between Tests and the ApiClient. They know HOW to call each API: which endpoint, what auth, whether to encrypt, how to build the payload. Tests only know WHAT business operation to perform.

WHY: REASONING: Without a Service Layer, every test file contains: build payload + encrypt + get token + add headers + call endpoint + handle errors. If the encryption key parameter changes, you edit 20 test files. With a Service Layer, you edit ONE service method.

HOW: Here is the mental model:

```
// WITHOUT Service Layer (bad &mdash; test knows too much)
@Test void testCreateCustomer() {
    String token = TokenManager.getAccessToken();
    String json = new ObjectMapper().writeValueAsString(request);
    String encrypted = EncryptionUtil.encrypt(json, key);
    EncryptedRequestWrapper wrapper = new EncryptedRequestWrapper(encrypted, key);
    given().header('Authorization', 'Bearer '+token).body(wrapper).post('/customer/create');
}

// WITH Service Layer (good &mdash; test knows only business intent)
@Test void testCreateCustomer() {
```

```
Response response = customerService.createRandomCustomer(encKey);  
validate(response).validateSuccess().assertAll();  
}
```

THINK: REASONING TEST: Read both tests above. Which one tells you the **BUSINESS PURPOSE** faster? The second one. The first one buries the purpose under HTTP plumbing. Tests should read like specifications: 'Create a customer, validate it succeeded.' How the customer gets created is the Service's job.

13. Response Validation — The Fluent Chain

WHAT: ResponseValidator.java provides a fluent (chainable) interface for validating API responses. It uses Soft Assertions internally, meaning ALL validations run even if some fail. At the end, assertAll() reports every failure together.

WHY: REASONING: Imagine validating 8 things about a Customer Create response. With hard assertions, the first failure stops everything — you fix it, re-run, find the next failure, fix it, re-run... 8 cycles. With ResponseValidator's soft assertions, ONE run shows you ALL 8 problems at once.

HOW: Step-by-step flow:

- **Step 1:** validate(response) — creates ResponseValidator wrapping the Response + a new SoftAssertions instance.
- **Step 2:** .validateSuccess() — adds two checks to SoftAssertions: statusCode == 200 AND body.status == 1.
- **Step 3:** .validateFieldNotNull('data[0].customerId') — adds: jsonPath value at that path is not null.
- **Step 4:** .validateFieldEquals('data[0].status', 'ACTIVE') — adds: field equals expected value.
- **Step 5:** .assertAll() — triggers ALL collected assertions. Reports every failure with clear messages.

TIP: The fluent chain returns 'this' from every method, which is why you can chain: validate(r).validateA().validateB().assertAll(). This is the Builder/Fluent pattern.

14. Data Generation — Never Hardcode Again

WHAT: DataGenerator.java uses JavaFaker (Indian locale) to generate random but realistic test data: Indian names, 10-digit mobile numbers starting with 6-9, valid email formats, Indian addresses, amounts, order references.

WHY: REASONING: Hardcoded data causes failures: (1) Run test twice → 'duplicate merchantCustomerId' error (2004). (2) Switch environments → data already exists. (3) Parallel tests → data collision. Random data makes every test run independent and repeatable.

HOW: Every generation method produces unique, valid-format data:

```
DataGenerator.merchantCustomerId() // 'MCUST-a7f2b39e' &mdash; unique every call
DataGenerator.indianMobile() // '9876543210' &mdash; starts with 6-9, 10 digits
DataGenerator.email() // 'priya.sharma.x8k2@testmail.com' &mdash; valid format
DataGenerator.fullName() // 'Arun Patel' &mdash; Indian names
DataGenerator.smallTestAmount() // 150 &mdash; random int 100-999
```

THINK: REASONING for Indian locale: SBlePay is an Indian payment gateway. Test data should match the domain. Indian mobile numbers have specific patterns (10 digits, starting with 6-9). Names like 'Priya Sharma' are more realistic for this system than 'John Smith'. This also helps catch locale-specific bugs.

15. JSON Templates — External Payload Management

WHAT: JSON template files stored in `src/main/resources/testdata/json/requests/` contain request body structures with `${placeholder}` tokens that get replaced with actual values at runtime.

WHY: REASONING: (1) Non-developers (manual testers, BAs) can modify request structures without touching Java code. (2) The JSON shape is visible without digging through Java builders. (3) Adding a new field to a request = edit a JSON file, not recompile. (4) Templates serve as documentation of what the API expects.

HOW: Step-by-step template injection:

- **Step 1:** `JsonUtil.readJsonTemplate("requests/customer_create.json")` loads the file as a String.
- **Step 2:** The template contains: `{"merchantCustomerId": "${merchantCustomerId}", ...}`
- **Step 3:** You provide a Map: `{'merchantCustomerId' → 'MCUST-abc123', 'customerName' → 'Priya'}`
- **Step 4:** `JsonUtil.injectValues()` replaces every `${key}` with the corresponding value.
- **Step 5:** Result: `{"merchantCustomerId": "MCUST-abc123", "customerName": "Priya", ...}`
- **Step 6:** For negative tests, `JsonUtil.removeField()` deletes a key entirely from the JSON.

THINK: REASONING: We use BOTH POJOs (primary) and templates (secondary). POJOs for type-safe happy-path tests. Templates for negative tests where you need to corrupt/remove fields easily. A POJO with `@Builder` always produces valid structures. Sometimes you WANT invalid structures — that is where templates with field removal shine.

Part III — Testing Concepts (Reasoning)

16. TestNG Fundamentals for API Testing

WHAT: TestNG is the test execution framework that runs your tests, manages their lifecycle, supports grouping, parallel execution, data providers, dependencies, listeners, and retry. It is the orchestrator that calls your `@Test` methods.

WHY: REASONING: JUnit 5 is an alternative, but TestNG offers: (1) Suite XML files for flexible test selection. (2) Groups (`@Test(groups='smoke')`) — run categories without separate classes. (3) `dependsOnMethods` — critical for E2E chains. (4) `DataProvider` for data-driven tests. (5) `ITestListener` for reporting hooks. JUnit can do most of this, but TestNG does it more naturally for API testing.

16.1 Annotation Lifecycle — What Runs When

Annotation	When It Runs	Our Usage	Reasoning
<code>@BeforeSuite</code>	Once before all tests in suite	Initialise <code>ApiClient</code> , <code>Services</code> , <code>Reports</code>	Expensive setup done once, shared by all tests
<code>@BeforeClass</code>	Once before the first test in a class	Class-specific setup (if needed)	Per-class initialisation
<code>@BeforeMethod</code>	Before EVERY <code>@Test</code> method	Reset state, clear soft asserts	Each test starts clean
<code>@Test</code>	The actual test	Business assertions	Where your validation lives
<code>@AfterMethod</code>	After EVERY <code>@Test</code> method	Log result to report	Capture pass/fail regardless of exceptions
<code>@AfterSuite</code>	Once after all tests complete	Flush reports, cleanup	Finalise HTML report file

16.2 Groups — Categorise Without Duplicating

WHAT: Groups tag tests with categories. One test can belong to multiple groups. Suite XMLs select which groups to run.

```
@Test(groups = {'regression', 'negative'}) // belongs to BOTH groups
void testMissingCustomerName() { ... }
```

When you run the 'smoke' suite, only tests tagged 'smoke' execute. When you run 'regression', everything tagged 'regression' OR 'negative' OR 'boundary' executes. No duplicate test code.

17. Soft vs Hard Assertions — Complete Comparison

17.1 Hard Assert (Traditional)

WHAT: Fails IMMEDIATELY on the first assertion failure. Remaining assertions in the method do NOT execute.

```
Assert.assertEquals(statusCode, 200); // if this fails...
Assert.assertNotNull(customerId); // ...this NEVER runs
```

```
Assert.assertEquals(status, 'ACTIVE'); // ...this NEVER runs
```

Use when: subsequent assertions are meaningless without the first passing. Example: if status code is 500, checking body fields is pointless.

17.2 Soft Assert (Our Approach)

WHAT: Collects ALL assertion failures, then reports them together when you call `assertAll()`. Every assertion runs regardless of previous failures.

```
SoftAssertions soft = new SoftAssertions();  
soft.assertThat(statusCode).isEqualTo(200); // if this fails...  
soft.assertThat(customerId).isNotNull(); // ...this STILL runs  
soft.assertThat(status).isEqualTo('ACTIVE'); // ...this STILL runs  
soft.assertAll(); // reports ALL failures at once
```

THINK: REASONING: In API testing, a single response often has 5-10 things to validate. If the server returns wrong data in 3 fields, you want to know about ALL 3 in one run — not discover them one at a time across 3 separate runs. Soft assertions save time and give a complete quality picture.

18. Test Groups, Suites & Parallel Execution

18.1 Suite XML Files — The Execution Playbook

WHAT: TestNG suite XMLs define WHICH tests run, in WHAT order, with WHAT parallelism. They are the deployment configuration for your test runs.

Suite	File	Contains	When to Run	Parallelism
Smoke	testng-smoke.xml	1-2 happy paths per API	After every deployment	None (sequential, fast)
Regression	testng-regression.xml	All positive + negative + boundary	Nightly or per PR	3 threads (test-level)
E2E	testng-e2e.xml	Complete payment flow chain	Pre-release sign-off	None (must be sequential)

THINK: REASONING: E2E tests *MUST* be sequential (`parallel=false`) because each step depends on data from the previous step (`customerId` → `orderId` → `orderHash`). Regression tests *CAN* be parallel because each test creates its own data. Wrong parallelism setting = random failures that are impossible to debug.

19. Retry & Flaky API Handling

WHAT: RetryAnalyzer automatically re-runs a failed test up to 2 times with exponential backoff (wait 2s, then 4s). RetryTransformer applies this to ALL tests globally without needing `@Test(retryAnalyzer=...)` on each method.

WHY: REASONING: Shared test environments are unstable. A 503 error at 2:00 AM during nightly regression should not fail the entire suite if the server recovers 3 seconds later. Retry with backoff handles transient failures while still catching real bugs (a real bug fails consistently across all retries).

HOW: Step-by-step retry flow:

- **Attempt 1:** Test runs. If it passes → done. If it fails → check retry count.
- **Retry decision:** `retryCount (0) < maxRetry (2)`? YES → wait $2^1 = 2$ seconds.
- **Attempt 2:** Test re-runs. If it passes → done (marked as PASS with retry note). If fails → check again.
- **Retry decision:** `retryCount (1) < maxRetry (2)`? YES → wait $2^2 = 4$ seconds.
- **Attempt 3:** Test re-runs. If it passes → done. If fails → `retryCount (2) = maxRetry (2)`. GIVE UP. Mark as FAILED.

THINK: REASONING for exponential backoff: If the server is overloaded, retrying immediately adds MORE load. Waiting progressively longer gives the server time to recover. 2s → 4s is gentle. Some systems use 1s → 2s → 4s → 8s → 16s for production retries.

20. E2E Flow — Chain APIs Like Production

WHAT: The PaymentFlowE2ETest executes the complete business flow: Access Token → Customer Create → Order Create → Order Status Update → Transaction Token → Booking → Payment Initiation. Each step passes dynamic data (IDs, tokens, hashes) to the next step.

WHY: REASONING: Individual API tests prove each API works in isolation. E2E tests prove they work TOGETHER. A Customer Create might return a valid customerId, and an Order Create might work with hardcoded data, but do they work when the Order uses the real customerId from the real Customer? Only E2E answers this.

20.1 Data Flow Across Steps

Step	Produces	Consumed By	Variable
Access Token	JWT token	Steps 2-6 (via TokenManager cache)	Cached internally
Customer Create	customerId	Order Create (optional)	customerId
Order Create	orderRefNumber	Order Status Update	orderRefNumber
Order Create	sbiOrderRefNumber	Order Status Update	sbiOrderRefNumber
Order Create	orderHash	TXN Token, Booking	orderHash
TXN Token	txnToken	Booking, Payment	Cached in TokenManager
TXN Token	hval	Payment encryption	Cached in TokenManager

THINK: REASONING: The dependsOnMethods annotation ensures if Step 3 (Order Create) fails, Steps 4-7 are SKIPPED (not FAILED). This is correct — they did not fail, they were not attempted because their prerequisite was not met. The report shows exactly where the chain broke.

Part IV — Infrastructure (Reasoning)

21. Logging — Seeing Without Exposing

WHAT: Log4j2 provides enterprise logging with configurable levels (DEBUG, INFO, WARN, ERROR), multiple outputs (console + file + error-only file), rolling archives (max 10MB per file, keep 5 archives), and pattern formatting with timestamps and thread names.

WHY: REASONING: When a test fails at 3 AM in the nightly run, you cannot attach a debugger. The logs are your ONLY window into what happened. Without structured logging, you have `System.out.println` scattered everywhere with no timestamps, no levels, no file output. With Log4j2, you open `target/logs/automation.log` and trace the exact request/response that failed.

21.1 Sensitive Data Masking — Sten by Sten

WHAT: `RequestLoggingFilter` intercepts every outgoing request and masks sensitive data BEFORE logging it. Card numbers show first 6 + last 4 digits. CVV is replaced with `***`. Bearer tokens show only the first 10 characters.

WHY: REASONING: Logs are often stored in shared systems (Jenkins, ELK, Splunk). If a full card number appears in logs, that is a PCI-DSS violation — a compliance and security risk. Masking ensures even if logs are accessed by unauthorised people, sensitive data is not exposed.

HOW: The filter applies regex patterns:

```
'4111111111111111' &rarr; '411111*****1111' (card masking)
'Bearer eyJhbGciOiJ...' &rarr; 'Bearer eyJhbGciOi***MASKED***' (token masking)
'cardSecurityCode':'123' &rarr; 'cardSecurityCode':'***' (CVV masking)
```

22. Reporting — Extent Reports Integration

WHAT: `ExtentManager` creates thread-safe Extent Reports that produce a rich HTML file with a dashboard (pie charts, pass/fail/skip counts), detailed test logs (request/response for every API call), categories (smoke, regression, e2e), environment info, and execution timeline.

WHY: REASONING: Your manager and stakeholders will NEVER read console output or TestNG XML. They need a visual report they can open in a browser. Extent Reports provides that with zero manual effort — the `TestListener` hooks into test lifecycle events and populates the report automatically.

HOW: The flow is: `TestListener.onTestStart()` creates a node → during the test, `logToReport()` adds request/response details → `TestListener.onTestSuccess/Failure()` marks the node → `@AfterSuite` calls `ExtentManager.flush()` to write the HTML file.

23. CI/CD — Jenkins, GitHub Actions, Docker

WHAT: Three CI/CD configurations are provided: `Jenkinsfile` (for Jenkins), `api-tests.yml` (for GitHub Actions), and `Dockerfile` (for containerised execution). Each one compiles the framework, runs the selected test suite against the selected environment, and publishes reports.

WHY: REASONING: A framework that only runs on your laptop is not enterprise-grade. Enterprise means: (1) Runs automatically on every code change (push/PR). (2) Runs on schedule (nightly regression). (3) Secrets never appear in code (injected from CI/CD vault). (4) Reports are automatically published and archived.

23.1 Docker — Why Containerise Tests?

WHAT: The Dockerfile packages the framework into a Docker image with JDK 17 + Maven + all dependencies. You run tests by starting a container with environment variables.

WHY: REASONING: 'It works on my machine' is the #1 problem in testing. Docker ensures the EXACT same Java version, Maven version, and dependency versions run everywhere — your laptop, CI server, colleague's machine. If it passes in Docker, it passes everywhere.

Part V — Every Framework File Explained

This section provides a reasoning-focused explanation for every file in the framework. For each file: WHAT it does, WHY it exists, and HOW it connects to the rest of the framework.

Maven Build Configuration

File: pom.xml

WHAT: Declares all 20+ dependencies (REST Assured, TestNG, Jackson, Lombok, Log4j2, etc.), configures plugins (Surefire, Failsafe, JaCoCo), and defines profiles for environment and suite switching.

WHY: Without pom.xml, you cannot compile or run anything. It is the project's DNA — every library version, every plugin, every build behaviour is controlled here.

HOW: Maven reads this file, downloads dependencies to ~/.m2/repository, configures the classpath, and executes test goals. Profiles are activated via -P flags.

Type-Safe Configuration Interface

File: src/main/java/.../config/FrameworkConfig.java

WHAT: A Java interface annotated with @Config.Sources. Each method maps to a property key. Owner library creates a runtime proxy that reads from property files.

WHY: Eliminates string-based property access. config.baseUrl() is compile-safe; Properties.getProperty('base.url') compiles fine but fails at runtime with a typo.

HOW: Owner library creates a dynamic proxy implementing this interface. When you call baseUrl(), it looks up 'base.url' in the loaded property files.

Configuration Singleton Provider

File: src/main/java/.../config/ConfigFactory.java

WHAT: Uses the Singleton pattern to create exactly ONE FrameworkConfig instance. Thread-safe via double-checked locking.

WHY: Creating multiple config instances wastes memory and can cause inconsistencies. Singleton ensures everyone reads the same configuration.

HOW: First call creates the instance via ConfigFactory.create(). Subsequent calls return the cached instance.

Centralised Constants

File: src/main/java/.../constants/ApiConstants.java

WHAT: Contains all magic strings: endpoint paths, header names, error codes, status values, timeouts, JSON path expressions.

WHY: If endpoint /customer/create changes to /customer/v2/create, you change ONE constant. Without this, you are through 30 files.

HOW: Organised as nested static classes: Endpoints, Headers, ErrorCodes, Status, JsonPaths, Timeouts. Import statically for clean code.

Central HTTP Client

File: src/main/java/.../core/client/ApiClient.java

WHAT: Wraps all REST Assured operations. Creates base RequestSpecification with baseUri, contentType, and logging filters. Provides typed HTTP methods: post(), get(), postWithAccessToken(), postWithTxnToken().

WHY: Single point of control for all HTTP communication. Add a global header once here and every API call gets it. Change timeout here and it applies everywhere.

HOW: Constructs RequestSpecification via RequestSpecBuilder. Injects auth tokens from TokenManager. Adds X-Correlation-ID for traceability. Returns raw Response.

Dual Token Manager

File: src/main/java/.../auth/TokenManager.java

WHAT: Manages ACCESS and TRANSACTION tokens with thread-safe caching. Lazy initialisation — tokens generated on first use. Auto-refresh when expired.

WHY: Without caching: 31 tests = 31 token API calls. With caching: 31 tests = 1 token call. Thread-safety is critical because regression suite runs with 3 parallel threads.

HOW: Uses synchronized blocks and ConcurrentHashMap. Checks token expiry before returning cached value. Includes a 60-second buffer before actual expiry.

AES Encryption/Decryption

File: src/main/java/.../encryption/EncryptionUtil.java

WHAT: Implements AES/CBC/PKCS5Padding encryption. Derives 256-bit key from input string via SHA-256. Generates random IV. Returns Base64-encoded ciphertext.

WHY: SBlePay requires encrypted payloads for sensitive APIs (customer create, order create, payment). Without this utility, services cannot call encrypted endpoints.

HOW: encrypt() derives key → generates IV → initialises Cipher → encrypts bytes → prepends IV → Base64 encodes. decrypt() reverses the process.

Customer Create POJO

File: src/main/java/.../models/requests/CustomerCreateRequest.java

WHAT: Java class with fields matching the Customer Create API request body. Lombok @Data generates getters/setters/toString. @Builder generates the builder pattern.

WHY: Type-safe request construction. IDE autocomplete shows all available fields. Compile-time error if you mistype a field name.

HOW: Jackson serialises this object to JSON automatically when passed to given().body(request). The @JsonInclude(NON_NULL) annotation omits null fields from JSON.

Encryption Wrapper POJO

File: src/main/java/.../models/requests/EncryptedRequestWrapper.java

WHAT: Generic wrapper with two fields: encryptRequest (the encrypted payload) and key (the hval). Used for all encrypted API calls.

WHY: Every encrypted API uses the same wrapper format. This POJO ensures consistent wrapping without string concatenation.

HOW: Service encrypts the inner payload, creates this wrapper, passes it to ApiClient. Jackson serialises it to {encryptRequest:'...', key:'...'}

Generic Response Model

File: src/main/java/.../models/responses/ApiResponse.java

WHAT: Models the standard SBlePay response structure: status (int), data (List), errors (List of ErrorDetail). ErrorDetail is a nested class with errorCode and errorMessage.

WHY: Deserialise any SBlePay response to this generic type. Check status field to determine success/failure. Access error codes programmatically.

HOW: response.as(ApiResponse.class) maps JSON to this object. Then apiResponse.getStatus() returns 1 or 0, apiResponse.getErrors() returns the error list.

Customer API Operations

File: src/main/java/.../services/CustomerService.java

WHAT: Provides methods: createRandomCustomer(), createCustomer(), createCustomerMissingField(), createCustomerWithField(), retrieveCustomer(), updateCustomerStatus(). Encapsulates payload building, encryption, and API calling.

WHY: Tests call one method and get a Response. They do not know about encryption, tokens, or HTTP. Changing the customer API (new field, different endpoint) requires editing only this file.

HOW: Each method: builds a POJO (or modifies JSON template for negative tests) → serialises to JSON → encrypts (if needed) → wraps → calls apiClient.postWithAccessToken() → returns Response.

Order API Operations

File: src/main/java/.../services/OrderService.java

WHAT: Provides: createRandomOrder(), createOrder(), createOrderMissingField(), updateOrderStatus(). Similar pattern to CustomerService.

WHY: Same reasoning as CustomerService. Encapsulates order-specific logic: random order reference generation, amount handling, status update payload construction.

HOW: For status update: builds OrderStatusUpdateRequest POJO, calls apiClient.postWithAccessToken(). No encryption needed for status update API.

Transaction & Payment Operations

File: src/main/java/.../services/TransactionService.java

WHAT: Provides: bookTransaction(), bookTransactionWithAccessToken() (for security negative test), initiateVisaDCPayment(). Handles the TOKEN SWITCH from ACCESS to TRANSACTION.

WHY: The booking and payment APIs require TRANSACTION token (not ACCESS). This service correctly uses postWithTxnToken() instead of postWithAccessToken(). The security test intentionally uses the wrong token to verify the server rejects it.

HOW: bookTransaction(): gets TXN token from TokenManager for the orderHash → calls /booking/{orderHash} with TXN Bearer token. initiateVisaDCPayment(): builds card payload → encrypts with hval → posts to /payment/initiation/DC.

Fluent Response Validator

File: src/main/java/.../validators/ResponseValidator.java

WHAT: Wraps a Response object and an AssertJ SoftAssertions instance. Each validate*() method adds an assertion and returns 'this' for chaining. assertAll() triggers all assertions.

WHY: Eliminates duplicate assertion code across tests.
validate(r).validateSuccess().validateFieldNotNull('data[0].id').assertAll() is reusable, readable, and complete.

HOW: Factory method validate(response) creates a new instance. Each method uses soft.assertThat() to register an assertion. The chain is terminated by assertAll() which throws if any assertion failed.

Request Logger with Masking

File: src/main/java/.../filters/RequestLoggingFilter.java

WHAT: Implements REST Assured's Filter interface. Intercepts outgoing requests, masks sensitive data (card numbers, CVV, tokens), and logs the sanitised version via Log4j2.

WHY: Every API call needs to be logged for debugging. But logs must not contain raw card numbers or tokens (security/compliance). This filter solves both needs.

HOW: Applied to the base RequestSpecification in ApiClient. Executes before every HTTP call. Uses regex patterns to identify and mask sensitive patterns.

Random Test Data Factory

File: src/main/java/.../utils/DataGenerator.java

WHAT: Static methods generating random Indian-locale test data: names, mobiles, emails, addresses, amounts, reference numbers. Also provides negative data helpers for boundary testing.

WHY: Hardcoded test data leads to: duplicate key errors on re-runs, environment conflicts, parallel test collisions. Random data makes every test execution independent.

HOW: Uses JavaFaker with Locale('en-IN') for Indian data. Custom logic ensures mobile numbers start with 6-9 and are exactly 10 digits. UUIDs ensure uniqueness.

JSON Template Utility

File: src/main/java/.../utils/JsonUtil.java

WHAT: Loads JSON files from classpath, replaces \${placeholder} tokens with runtime values, and provides field modification methods for negative testing (removeField, setFieldNull, setFieldValue).

WHY: Secondary payload management approach (after POJOs). Essential for negative tests where you need to produce intentionally malformed JSON that a POJO builder cannot create.

HOW: readJsonTemplate() reads file as String. injectValues() does String.replace() for each placeholder. removeField() uses Jackson to parse, remove a node, and re-serialise.

Exponential Backoff Retry

File: src/main/java/.../retry/RetryAnalyzer.java

WHAT: Implements TestNG IRetryAnalyzer. On test failure, checks if retry count < max (2). If yes, waits with exponential backoff and returns true (retry). If no, returns false (fail).

WHY: Handles transient API failures (503, timeouts) without manual intervention. A test that fails once due to a network blip passes on retry. A real bug fails all retries.

HOW: TestNG calls `retry()` when a test fails. Method increments counter, sleeps for $2^{\text{retryCount}}$ seconds, returns true. TestNG re-executes the test method.

Global Retry Applicator

File: `src/main/java/.../retry/RetryTransformer.java`

WHAT: Implements `IAnnotationTransformer`. TestNG calls it before every test method. It sets `RetryAnalyzer` on every `@Test` annotation automatically.

WHY: Without this, you would need `@Test(retryAnalyzer = RetryAnalyzer.class)` on every single test method. The transformer applies it globally — zero per-test configuration.

HOW: Registered in suite XML as a listener. TestNG calls `transform()` for each `@Test`. Method sets `annotation.setRetryAnalyzer(RetryAnalyzer.class)`.

Thread-Safe Report Singleton

File: `src/main/java/.../reports/ExtentManager.java`

WHAT: Creates and manages a single `ExtentReports` instance. Thread-safe via synchronized. Configures report title, system info (env, Java version, OS), and HTML output path.

WHY: Multiple test threads must write to the same report without corruption. Singleton with synchronisation ensures thread safety.

HOW: First access creates `ExtentReports`, configures Spark reporter, sets system info. Tests call `createTest()` to add nodes. `flush()` at suite end writes the HTML file.

TestNG Lifecycle Hooks

File: `src/main/java/.../listeners/TestListener.java`

WHAT: Implements `ITestListener`. Hooks into test events: `onStart` (create Extent node), `onSuccess` (mark PASS), `onFailure` (mark FAIL with exception), `onSkip` (mark SKIP), `onFinish` (flush report).

WHY: Connects test execution events to Extent Reports automatically. No manual `report.log()` needed in test methods.

HOW: Registered in suite XML. TestNG calls the appropriate method at each lifecycle point. The listener creates `ExtentTest` nodes and logs results.

Base Custom Exception

File: `src/main/java/.../exceptions/FrameworkException.java`

WHAT: `RuntimeException` subclass. Parent of `ApiException` and `EncryptionException`. Provides constructors for message and cause.

WHY: Custom exceptions give meaningful error messages. 'EncryptionException: Failed to encrypt payload with key length 24' is infinitely more useful than 'NullPointerException'.

HOW: Thrown by framework classes when something goes wrong. Tests can catch specific exception types for different handling.

Test Parent Class

File: `src/test/java/.../tests/BaseTest.java`

WHAT: All test classes extend this. @BeforeSuite initialises ApiClient, Services, and ExtentManager. Provides helper methods: validate(), logToReport(), getEncryptionKey(). @AfterSuite flushes reports.

WHY: DRY principle. Setup code lives here once, not in every test class. If you add a new service, add it here and all test classes can use it immediately.

HOW: TestNG guarantees @BeforeSuite runs before any @Test. This is where the framework 'boots up'. Protected fields (apiClient, customerService, etc.) are accessible to all child test classes.

Token Generation Tests

File: src/test/.../smoke/AccessTokenTest.java

WHAT: 5 tests: valid token generation, missing API Key ID, missing API Key Secret, invalid credentials, response time SLA check.

WHY: Token generation is the foundation. If this fails, NOTHING works. Smoke suite runs these first. Covers both happy path and critical negative scenarios.

HOW: Each test calls the token endpoint directly (no service layer needed for this simple API). Validates JWT structure (contains dots), response time under SLA.

Customer API Tests

File: src/test/.../regression/CustomerTest.java

WHAT: 10+ tests covering: create with all fields, create with mandatory only, missing name/mobile/email, boundary values (name > 100 chars, mobile != 10 digits), retrieve valid/invalid, status updates (active/inactive/delete).

WHY: Comprehensive coverage of one API. Positive tests verify it works. Negative tests verify it fails correctly (right error code). Boundary tests verify limits are enforced.

HOW: Uses CustomerService for all API calls. ResponseValidator for all assertions. DataGenerator for unique test data per run.

Order API Tests

File: src/test/.../regression/OrderTest.java

WHAT: 8+ tests: create valid order, missing amount/currency, boundary values, status update to ATTEMPTED/CONFIRMED, null/invalid status.

WHY: Same pattern as CustomerTest but for the Order API. Each test is independent and creates its own data.

HOW: Uses OrderService. Tests status transitions: CREATED → ATTEMPTED is valid, but invalid statuses should return error codes.

End-to-End Payment Flow

File: src/test/.../e2e/PaymentFlowE2ETest.java

WHAT: 8 sequential tests: Generate Access Token → Create Customer → Create Order → Update Status → TXN Token → Booking → Payment DC → Security Negative Test.

WHY: Integration testing. Proves the entire chain works together with real dynamic data passing. The security test verifies ACCESS token cannot be used for TRANSACTION operations.

HOW: Instance variables pass data between steps. dependsOnMethods ensures order. If Step 3 fails, Steps 4-7 are SKIPPED, not FAILED. The report shows exactly where the chain broke.

Configuration & Resource Files

Default Configuration

File: config/config.properties

WHAT: Key-value pairs for base URL, credentials, timeouts, retrv settings, loading flags, report settings.

WHY: Centralised configuration. Change one value here instead of editing Java code.
Environment-specific files override these defaults.

HOW: Owner library loads this file first, then layers the env-specific file. Keys in the env file override matching keys here.

SIT Environment Overrides

File: environments/sit.properties

WHAT: Overrides base URL, credentials, and timeouts for the SIT (System Integration Testing) environment.

WHY: SIT has different URLs and credentials than UAT or PROD. This file provides SIT-specific values without changing the default config.

HOW: Activated when -Denv=sit is passed (via Maven profile or command line). Owner loads it via the \${env} variable in @Config.Sources.

Logging Configuration

File: log4j2.xml

WHAT: Defines three appenders: Console (coloured output), FileAppender (rolling 10MB files), ErrorAppender (errors only). Configures log levels per package.

WHY: Different outputs for different needs. Console for local development. File for CI/CD analysis. Error file for quick triage of failures.

HOW: Log4j2 auto-detects this file on classpath. monitorInterval=30 means changes are picked up without restart.

Smoke Suite Definition

File: testng-smoke.xml

WHAT: Selects only the happy-path tests from each test class. Runs sequentially for speed.

WHY: Quick health check after deployment. If smoke fails, stop and investigate — do not run full regression.

HOW: Maven profile 'smoke' points Surefire plugin to this file as the suiteXmlFile parameter.

Regression Suite Definition

File: testng-regression.xml

WHAT: Includes ALL test groups: regression, smoke, negative, boundary. Runs with 3 parallel threads.

WHY: Full coverage for nightly runs or PR validation. Parallel execution reduces total time from 15 min to ~5 min.

HOW: The parallel='tests' attribute means each <test> block runs in its own thread. Tests within a block run sequentially.

Customer Request Template

File: customer_create.json

WHAT: JSON template with `${placeholder}` tokens for all customer fields.

WHY: Non-developers can modify the request structure. Templates serve as API documentation. Used for negative tests via field removal.

HOW: `JsonUtil.readJsonTemplate()` loads it. `injectValues()` replaces placeholders. `removeField()` creates negative test variants.

Success Response JSON Schema

File: success_response.json (Schema)

WHAT: Defines the contract: status must be integer 1. data must be a non-empty array.

WHY: Structural validation catches breaking API changes. If the server adds/removes/renames a field, schema validation fails before your assertions even run.

HOW: `ResponseValidator.validateJsonSchema()` loads this file and uses REST Assured's `matchesJsonSchemaInClasspath()` matcher.

Jenkins Pipeline

File: Jenkinsfile

WHAT: Declarative pipeline with parameterised builds (ENV + SUITE), credential injection from Jenkins vault, report publishing, and Slack notifications.

WHY: Automates the entire test lifecycle in Jenkins. No manual steps. Secrets never appear in code.

HOW: Jenkins reads this file from the repo. Parameters appear as dropdowns when triggering a build. Credentials are injected as environment variables.

GitHub Actions Workflow

File: api-tests.yml

WHAT: Triggers on push/PR/schedule/manual dispatch. Sets up JDK, runs Maven with secrets from GitHub vault, uploads reports as artifacts.

WHY: Native CI/CD for GitHub repositories. Test results appear directly in PR checks. Artifacts are downloadable for 30 days.

HOW: GitHub reads `.github/workflows/*.yml` automatically. The strategy matrix could be extended to run against multiple environments simultaneously.

Container Configuration

File: Dockerfile

WHAT: Multi-stage build: Stage 1 downloads dependencies and compiles. Stage 2 (slim) runs tests. Entry point accepts ENV, SUITE, and credential environment variables.

WHY: Consistent execution environment everywhere. No dependency version conflicts between machines. Easy to run in any CI system that supports Docker.

HOW: docker build creates the image once. docker run starts a container, runs tests, and exits. Volume mount (-v) makes reports accessible on the host.

Git Ignore Rules

File: .gitignore

WHAT: Excludes: target/ (build output), IDE files (.idea/, *.iml), logs, secrets files (.env), OS files (.DS_Store).

WHY: Prevents committing generated files, IDE-specific config, and especially secrets. A committed .env file is a security incident.

HOW: Git reads this file and excludes matching paths from version control. Always review before first commit.

Part VI — Payload Management (8 Approaches)

This section compares all 8 payload management approaches with reasoning for when to use each one. Our framework uses **Approach 4 (POJO + Builder)** as **primary** and **Approach 6 (External JSON)** as **secondary**.

Approach 1: Hardcoded JSON String

WHAT: Embed JSON directly as a Java string with escape characters.

When to Use: Quick prototyping, debugging in IDE

Used By: Never in production frameworks

***THINK:** No type safety, unreadable escape characters, impossible to maintain, cannot inject dynamic values cleanly.*

Approach 2: HashMan

WHAT: Use Map<String, Object> with put() calls to build the payload.

When to Use: Small payloads, quick tests, when structure is unknown/dynamic

Used By: Startups, small teams, POCs

***THINK:** No compile-time key validation. `body.put('custmerName',...)` compiles but fails at runtime. No IDE autocomplete for keys.*

Approach 3: JSONObject (org.json)

WHAT: Use JSONObject/JSONArray to build JSON programmatically.

When to Use: Complex JSON manipulation, nested structure modification

Used By: Teams doing heavy JSON transformation

***THINK:** Same lack of type safety as HashMap. An extra dependency for something REST Assured + Jackson already handles.*

Approach 4: POJO + Builder (PRIMARY)

WHAT: Java classes with Lombok @Builder for type-safe construction.

When to Use: ALL positive/happy-path tests. Primary approach for enterprise frameworks.

Used By: Google, Amazon, Visa, Razorpay, JPMC, ThoughtWorks

***THINK:** Compile-time safety. IDE autocomplete. Refactor-friendly. Self-documenting. Jackson auto-serialises. This is the industry standard.*

Approach 5: Custom Builder Pattern

WHAT: Hand-written builder classes (without Lombok).

When to Use: When Lombok is not allowed (some enterprises restrict annotation processors)

Used By: Conservative enterprises

THINK: Same benefits as Approach 4 but with 5x more boilerplate code. Use Lombok instead if allowed.

Approach 6: External JSON Templates (SECONDARY)

WHAT: JSON files with \${placeholder} tokens, loaded and injected at runtime.

When to Use: Negative tests (field removal), data-driven tests, non-developer-editable payloads

Used By: EPAM, Publicis Sapient, Infosys

THINK: Non-developers can modify. Clear structure. But no compile-time validation of field names or types.

Approach 7: Template Engine (Mustache/Handlebars)

WHAT: Full template engine with conditional logic, loops, partials.

When to Use: Complex conditional payloads (if premium customer, include extra fields)

Used By: Niche cases, API mocking servers

THINK: Overkill for testing. Adds complexity and a learning curve. Simple placeholder replacement (Approach 6) covers 95% of needs.

Approach 8: Dynamic Payload Builder (Fluent)

WHAT: Fluent interface for building JSON dynamically: Payload.create().set('field','value').remove('other').

When to Use: Negative tests needing field-level manipulation, combined with POJO approach

Used By: Advanced teams

THINK: Complements POJOs for edge cases. Our JsonUtil.removeField() and setFieldValue() serve this purpose.

Part VII — Interview Questions & Anti-Patterns

Top Interview Questions with Reasoning-Based Answers

Q: How do you handle API authentication in your framework?

We use a dual-token system with a centralised TokenManager. ACCESS tokens are for merchant operations and cached with expiry tracking. TRANSACTION tokens are per-order and cached by orderHash. The TokenManager is thread-safe for parallel test execution. Tests never handle tokens directly — the Service Layer and ApiClient inject them automatically. This follows the Single Responsibility Principle.

Q: How do you manage test data?

We use JavaFaker with Indian locale for random but realistic data. Every test generates unique data (unique merchantCustomerId via UUID), so tests are independent, repeatable, and safe for parallel execution. No hardcoded data means no duplicate key errors on re-runs and no environment conflicts.

Q: How do you validate API responses?

We use a custom ResponseValidator with fluent chain syntax backed by AssertJ soft assertions. This validates: HTTP status, business status, specific fields, error codes, patterns, schemas, and response time. All failures are collected and reported together via `assertAll()`, giving a complete quality picture in one run.

Q: How do you handle flaky tests?

We implement retry with exponential backoff via TestNG IRetryAnalyzer. Failed tests retry up to 2 times with increasing wait (2s, 4s). This handles transient issues (503, timeouts) while real bugs still fail consistently across all retries. The retry is applied globally via IAnnotationTransformer.

Q: How do you run tests across environments?

Maven profiles set the -Denv flag. The Owner library loads environment-specific property files that override defaults. Switching from SIT to UAT is one flag change: `mvn test -Denv=UAT`. Secrets come from environment variables or CI/CD vault, never from property files.

Q: What is the Service Layer pattern and why use it?

The Service Layer sits between tests and the HTTP client. It encapsulates: payload construction, encryption, authentication, and endpoint knowledge. Tests call `customerService.createRandomCustomer()` and validate the response. If the API changes (new field, different encryption), only the Service changes — zero test modifications. This is Separation of Concerns.

Q: How do you handle encrypted request payloads?

A centralised EncryptionUtil handles AES/CBC/PKCS5Padding encryption. Services build plain payloads, encrypt them, and wrap in EncryptedRequestWrapper. Tests are completely unaware of encryption. Changing the algorithm means editing one file.

Q: Explain your reporting strategy.

ExtentReports generates rich HTML reports with dashboards, request/response logs, categories, and environment info. A TestNG ITestListener hooks into lifecycle events (start, pass, fail, skip) and populates

the report automatically. Reports are published as CI/CD artifacts for stakeholder access.

Anti-Patterns to Avoid

Tests that know HTTP details

Problem: Tests should call Service methods, not construct HTTP requests directly. If a test contains `given().header().body().post()`, the Service Layer is being bypassed.

Solution: Move HTTP logic to Service classes. Tests should contain only business assertions.

Hardcoded test data

Problem: Using 'CUST001' in every test causes duplicate errors on re-run and parallel conflicts.

Solution: Use DataGenerator for unique data every run.

Only checking HTTP status code

Problem: HTTP 200 does not mean success in SBlePay. The business status field must also be checked.

Solution: Always validate both: `.validateStatusCode(200).validateSuccess()`

Using hard assertions for multiple checks

Problem: First failure hides all subsequent issues. Fixing one at a time wastes cycles.

Solution: Use ResponseValidator (soft assertions). See all failures in one run.

Thread.sleep() instead of polling

Problem: Arbitrary waits are either too long (slow tests) or too short (flaky tests).

Solution: Use Awaitility or retry with condition checking.

Committing secrets

Problem: Credentials in property files committed to Git are a security incident.

Solution: Use environment variables or CI/CD vault. Placeholder values in committed files.

No correlation IDs

Problem: When a test fails, you cannot find the corresponding server-side log entry.

Solution: Our ApiClient adds X-Correlation-ID to every request automatically.

God test classes

Problem: One test class with 50 methods covering 5 different APIs.

Solution: One test class per API area. CustomerTest, OrderTest, PaymentFlowE2ETest.

End of Guide — 59 files, 31+ tests, 9 APIs, complete CI/CD. Every concept explained with WHAT, WHY, HOW reasoning.

Framework version 1.0 • REST Assured 5.4.0 • TestNG 7.9.0 • Java 17